ChatGPT

Audio, Speech, and Music ML Systems Ecosystem Map 2026

Ecosystem overview

Executive summary

Area	Most practical open stack right now	Why it wins in practice	Deployment reality (self-host / latency)	Common trap	Verdict label
Offline ASR (general)	faster-whisper (CTranslate2) with Whisper large-v3 / large-v3-turbo	Strong quality + very mature quantization + easy batching; huge community footprint. [1]	Self-host: high. Local: high. CPU-only: viable for small/quantized. GPU: strongly recommended for high throughput. Latency: batch/offline preferred; can be near-real-time with chunking. [2]	Treating it as "streaming ASR" without designing for streaming (chunking, overlap, partial hypotheses). [3]	strongest open baseline
Low-latency / true streaming ASR	Sherpa (PyTorch) + transducer/CTC models; NeMo cache-aware streaming Conformer/FastConformer	Designed around streaming constraints (right-context limits, caching, transducer decoding). [4]	Self-host: high (but model + integration work). Latency: near-real-time to real-time on commodity GPU; CPU real-time mostly for smaller models. [4]	Assuming "Whisper-like" seq2seq behaves like streaming; it generally doesn't without redesign. [3]	production-friendly (if you pick streaming-native models)

Area	Most practical open stack right now	Why it wins in practice	Deployment reality (self-host / latency)	Common trap	Verdict label
Edge / embedded ASR	sherpa-ncnn / sherpa-onnx (ONNXRuntime), whisper.cpp (ggml/gguf)	Good portability; quantized inference; supports mobile/ARM paths. [5]	Real-time possible on strong ARM for small models; for large models expect "CPU-only but slower than real-time" unless heavily quantized. [6]	Underestimating memory bandwidth + thermal throttling; "it runs" ≠ "it runs continuously". [7]	best for streaming/ offline edge
Russian ASR	GigaAM / GigaAM-v3 (Conformer + SSL objectives)	Explicitly trained for Russian; strong author-reported WER on Russian sets; includes CTC + RNN-T variants. [8]	Self-host: medium-high (Python stack, checkpoints; ONNX export exists in ecosystem). Latency: depends on model size/ decoder; RNN-T is streaming-friendly in principle. [9]	Confusing "multilingual coverage" with "Russian accuracy in noisy/telephony domains". [10]	strong Russian ASR choice
Speaker diarization	pyannote pipelines (3.1; newer community-1) + NeMo diarization pipeline	Most plug-and-play open diarization; active updates; NeMo offers full pipeline components. [11]	Self-host: medium (HF gating / tokens; GPU helpful). Latency: batch/offline preferred; "real-time diarization" is harder than it sounds. [12]	Expecting stable "speaker IDs"; diarization labels are session-local (SPEAKER_00, …). [13]	production-friendly (with caveats)

Area	Most practical open stack right now	Why it wins in practice	Deployment reality (self-host / latency)	Common trap	Verdict label
Speech emotion recognition	GigaAM-Emo; SpeechBrain wav2vec2 emotion models; openSMILE features as baseline	Easy inference exists, but reliability is brittle across domains/ languages; best used as weak signal. [14]	Self-host: easy; latency: clip-level near-real-time. But "truthfulness" is low. [15]	Treating outputs as objective psychological state, or as stable across cultures/ languages. [16]	good enough but caution
General audio tagging / events	PANNs (CNN14 family), YAMNet, AST, HTS-AT, BEATs	Mature AudioSet transfer; transformer models often stronger but heavier. [17]	Large library scanning: CNN or efficient transformer; CPU-only feasible for small nets. [18]	Confusing "audio event" tags with "music semantics" (genre/ mood). [19]	production-friendly (choose by compute budget)
Music understanding / MIR embeddings	MERT, MusicFM, CLAP (audio-text), OpenL3 (older)	Modern music reps improve retrieval/ similarity; CLAP enables text-to-audio retrieval but is not "music theory aware." [20]	Batch scanning: feasible; GPU recommended for big collections. Latency: batch/offline preferred. [21]	Expecting a single embedding to solve tags + beat + chord + structure. [22]	promising but niche (depends on task)

Short practical takeaways (minimal prose)

Streaming ASR is a different engineering problem than offline transcription: you need time-synchronous decoding, boundary handling, partial hypotheses, and latency budgets; many "great offline" seq2seq models are simply not built for that. [23]

For self-hosting, the biggest practical differentiator is not the paper architecture—it's the inference stack: quantization format, runtime portability, and batching support dominate cost/latency. [24]

Emotion recognition is a weak signal system: treat it like noisy metadata, not ground truth. [16]

Common confusion / misconceptions

Whisper-family ≠ streaming ASR stack (without additional design). [25]

Diarization ≠ speaker identification (names); diarization is "who spoke when," not "who is Alice." [26]
Audio tagging models (AudioSet) are not music-specialized MIR models; they often capture events more than musical structure. [27]

One-page ecosystem map

Category	What it does	Dominant model styles (2026 reality)	Practical "current ecosystem" examples	What people commonly confuse it with
ASR / STT	Audio → text (timestamps optional)	Streaming: Conformer + CTC/RNN-T; Offline: seq2seq encoder-decoder (Whisper-like) + batching/quantization. [28]	Sherpa (E2E deployment framework for CTC/transducer), sherpa-onnx, whisper.cpp, faster-whisper, NeMo streaming Conformer/FastConformer, WeNet, Vosk. [29]	"If it's accurate offline, it's good for streaming." [30]
Speech emotion recognition	Speech → emotion label(s) / dimensions	SSL backbone (wav2vec2/HuBERT/WavLM) + classifier; sometimes hand-crafted features (openSMILE) + ML. [31]	GigaAM-Emo; SpeechBrain wav2vec2-IEMOCAP; openSMILE baselines. [32]	"Emotion labels are objective and stable across domains/languages." [16]
Speaker recognition / verification	Voice → speaker embedding; same/different speaker	ECAPA-TDNN / x-vector families; contrastive losses; embeddings + cosine/PLDA. [33]	SpeechBrain ECAPA-TDNN; NeMo TitaNet; (others exist, but these are common open baselines). [34]	"Speaker verification solves diarization" (it doesn't; diarization needs segmentation/clustering). [35]
Diarization / segmentation	"Who spoke when" (turns; overlap)	Cascaded: VAD + embeddings + clustering + resegmentation; newer "end-to-end diarization" exists but less deployment-mature. [36]	pyannote pipelines (3.1, community-1); NeMo diarization pipeline (MarbleNet VAD + TitaNet + MSDD). [37]	"Diarization gives stable identities across recordings." [13]

Category	What it does	Dominant model styles (2026 reality)	Practical "current ecosystem" examples	What people commonly confuse it with
General audio classification	Audio clip → event/scene labels	CNNs on log-mel; transformers on spectrogram patches; AudioSet pretraining dominates. [38]	YAMNet, PANNs, AST, HTS-AT, BEATs. [39]	"Audio event tagging = music tagging." [40]
Music tagging / genre / mood / instrument	Music clip → multi-label tags or genre/ mood	Music CNN tagging (older but light); modern music transformers / SSL music reps; multi-label heads. [41]	Musicnn; MERT; MusicFM; also PANNs/AST fine-tuning for music tags. [42]	"Genre is a unique ground truth label." [43]
Rhythm / beat / tempo analysis	Beats/ downbeats/ tempo/meter	Classic DSP (onset/ tempo DP) + ML beat trackers (CRNN + post filters). [44]	librosa beat_track; madmom RNN processors; Essentia beat tracking; BeatNet. [45]	"Tempo estimation = beat tracking" (beat positions are harder than a single BPM). [46]
Audio embeddings / representation	Audio → vector embedding for retrieval/ transfer	Speech SSL (wav2vec2/HuBERT/ XLS-R/WavLM); audio transformers (BEATs); audio-text (CLAP); older baselines (OpenL3/ VGGish). [47]	BEATs; CLAP; WavLM; XLS-R; OpenL3; VGGish; MusicFM/ MERT for music. [48]	"One embedding solves all audio tasks equally well." [49]
Toolkits vs models vs wrappers	Training/ inference scaffolding	Toolkits (train + recipes) vs deployment frameworks vs runtime backends. [50]	SpeechBrain, NeMo, ESPnet, WeNet (toolkits); sherpa/ sherpa-onnx (deployment); faster-whisper/whisper.cpp (runtime wrappers); ONNXRuntime/ CTranslate2/ggml/ ncnn (runtimes). [51]	"Toolkit name = one specific model." [52]

Short practical takeaways (minimal prose)
When you evaluate an "ecosystem," separate: (1) base model family, (2) training toolkit, (3) runtime backend, (4) pipeline glue (VAD, diarization, alignment). Most real failures happen in (3) and (4). [53]

Common confusion / misconceptions
"Open-source model" ≠ "ungated weights" (pyannote pipelines often require HF acceptance/token). [54]
"Streaming supported" in a repo ≠ "production streaming UX" (partial results, endpointing, barge-in, jitter). [55]

Master comparison table

Legend (applies across tables):
Latency/hardware entries are **documented** when explicitly stated in project docs or reproducible reports; otherwise they are **inferred** from architecture + typical deploy practice and labeled as such.

Name	Type	Primary domain	Main tasks	Architecture (short)	Core principle (short)	Streaming / offline / batch
Sherpa	Deployment framework (inference)	Speech	ASR (CTC, transducer)	E2E CTC & RNN-T inference stack in C++/ Python (PyTorch) [56]	"Deploy pre-trained E2E models" (not training) [56]	Streaming + offline (model-dependent) [56]
sherpa-onnx	Deployment framework + ONNX model runner	Speech + audio	ASR (streaming/ non), diarization, speaker ID/ verify, LID, tagging, VAD, etc. [57]	ONNX models + onnxruntime; multi-language bindings [58]	"Local, offline processing" with ONNXRuntime [59]	Streaming + offline; batch possible [60]
GigaAM / GigaAM-v3	Model family + Python package	Speech (Russian)	Russian ASR (CTC, RNN-T), emotion (Emo) [61]	Conformer-based foundation model; v3 uses HuBERT-CTC objective; fine-tuned heads for tasks [62]	Russian-focused pretraining + task finetuning [63]	Offline (CTC) + streaming-friendly (RNN-T) in principle [10]
SpeechBrain	Training + inference toolkit	Speech (broad)	ASR, speaker tasks, emotion, VAD, etc. [66]	PyTorch toolkit + recipes; integrates many pretrained backbones [67]	"Research + reproducible recipes + inference APIs" [68]	Depends (toolkit)

Name	Type	Primary domain	Main tasks	Architecture (short)	Core principle (short)	Streaming / offline / batch
Musicnn	Model + library	Music	Music tagging (multi-label)	CNN on mel spectrogram, music-tagging focused [70]	Supervised music tagging; efficient inference [71]	Offline/batch
Whisper family	Model family	Speech	Multilingual ASR + some translation	Seq2seq encoder-decoder transformer; long-form “sequential” decoding style in tooling [72]	Large-scale supervised training; strong robustness; not streaming-native [3]	Offline/batch primarily; streaming only via approximations [3]
faster-whisper	Runtime wrapper	Speech	Whisper inference optimized	Whisper via CTranslate2; supports quantization and fast decoding [73]	Replace PyTorch with optimized transformer runtime [74]	Offline/batch; chunking possible
whisper.cpp	Runtime + model format ecosystem	Speech	Whisper inference local	ggml/gguf quantized inference; C/C++ focus [78]	Aggressive portability + quantization for local inference [78]	Offline; some streaming-ish modes via chunking (inferred)
NeMo	Toolkit + model zoo	Speech	ASR, diarization, speaker, etc.	Conformer/FastConformer + CTC/RNN-T; cache-aware streaming; diarization pipeline components [81]	Industrial-grade recipes; streaming-first options [82]	Streaming + offline

Name	Type	Primary domain	Main tasks	Architecture (short)	Core principle (short)	Streaming / offline / batch
pyannote.audio pipelines	Toolkit + pretrained pipelines	Speech (speaker)	Diarization, VAD, segmentation	PyTorch pipelines; gated model access; newer community-1 pipeline exists [86]	End-to-end building blocks + ready pipelines [87]	Offline/batch
YAMNet	Pretrained model	General audio	Audio event classification	MobileNet-like CNN on log-mel; AudioSet classes [90]	Small model for broad audio taxonomy [91]	Offline; streamable in frames (inferred)
PANNs	Model family + inference libs	General audio	Tagging, SED transfer	CNNs (CNN14 etc); Wavegram-Logmel CNN; AudioSet pretraining [92]	Large-scale supervised pretraining for transfer [19]	Offline/batch; framewise SED possible
AST	Model family	General audio	Audio classification	ViT-style transformer over spectrogram patches [94]	“Pure attention” audio classifier [95]	Offline/batch
HTS-AT	Model family	General audio	Audio classification + detection	Hierarchical transformer + token-semantic module [96]	Hierarchical design to reduce compute, enable detection head [97]	Offline/batch
BEATs	Representation model	General audio	Audio embeddings + classification transfer	Transformer SSL for audio (acoustic tokenizer / masked prediction) [98]	Self-supervised audio pretraining for transfer [98]	Offline/batch embeddings

Name	Type	Primary domain	Main tasks	Architecture (short)	Core principle (short)	Streaming / offline / batch
CLAP (LAION / MS-CLAP)	Model family	Audio-text	Audio↔text embeddings; zero-shot labeling	Dual encoder (audio + text) contrastive learning [99]	Align audio/ text for retrieval/zero-shot classification [100]	Offline/batch embeddings
MERT	Representation model	Music	Music embeddings / tagging transfer	Transformer-based music representation model [104]	Music-focused self-supervised / pretraining for transfer [104]	Offline/batch
MusicFM	Representation model	Music	Music embeddings / retrieval	Foundation model for music representation [105]	Pretrained representations for music tasks [106]	Offline/batch
Essentia	Toolkit	Music/ MIR	Features + beat/tempo + descriptors	C++/Python library with many MIR algorithms [107]	Practical MIR feature extraction + some ML models in ecosystem [108]	Offline; some real-time capable depending feature [109]

Short practical takeaways (minimal prose)
Inference wrappers are often more decisive than model family: CTranslate2 (transformer inference + quantization) vs ONNXRuntime vs ggml/gguf vs ncnn all change what's "deployable." [111]
If you need diarization + word timestamps, recognize it's a pipeline (ASR + alignment + diarization), not one model. [112]

Common confusion / misconceptions
"Supports diarization" in a framework ≠ "best-in-class diarization model." [113]
"Zero-shot" audio labeling with CLAP depends heavily on prompts/class definitions and may not correlate with human judgments. [114]

Speech tasks deep dives

ASR / speech recognition

A. Main ASR table

Ratings:
Offline quality: High / Med / Low (for the model family in typical usage).
Russian support quality: Excellent / Good / Mixed / Weak (explicitly about Russian ASR practicality).

Model / family	Model or toolkit?	Core architecture	Training paradigm	Decoder style	Streaming support	Offline transcription quality	R s
Whisper (incl. turbo)	Model family	Seq2seq encoder-decoder transformer [3]	Supervised large-scale; turbo fine-tuned for speed with smaller decoder [3]	Autoregressive seq2seq	Not streaming-native; “chunking” approximations only [3]	High (offline)	M (c ic u la
faster-whisper	Runtime wrapper	Whisper via CTranslate2 [73]	Same as Whisper	Same	Same model limits; wrapper enables batching/ quantization	High	B tl tl b n ti p w c
whisper.cpp	Runtime + format	Whisper in ggml/gguf with quantization [78]	Same as Whisper	Same	Same model limits	High for small–mid; large depends on quantization	G C s m s tl ti

Model / family	Model or toolkit?	Core architecture	Training paradigm	Decoder style	Streaming support	Offline transcription quality	R s
Sherpa	Toolkit (deployment)	E2E CTC + transducer inference [56]	Uses pretrained E2E models; training elsewhere (icefall) [56]	CTC, RNN-T	Yes (transducer/ streaming models)	Depends on model	H u s tr
sherpa-onnx	Toolkit (deployment)	ONNX models via onnxruntime [116]	Exported models	CTC/RNN-T/ others (model-dependent)	Yes (streaming + non-streaming) [60]	Depends on model	G u s m
GigaAM / GigaAM-v3	Model family	Conformer-based foundation model; v3 HuBERT-CTC objective [62]	SSL pretrain + fine-tune for ASR heads [118]	CTC + RNN-T variants [10]	RNN-T variant streaming-friendly in principle [10]	High (Russian)	P s o d o
NeMo streaming Conformer/ FastConformer	Toolkit + model families	Conformer/ FastConformer + cache-aware streaming; hybrid CTC/ RNNT options [119]	Supervised; optionally SSL backbones elsewhere	CTC / RNN-T / Hybrid	Yes; cache-aware streaming designed for limited right context + caching [120]	High (for well-trained domain)	H (s fi
WeNet	Toolkit	Two-pass unified streaming/ non-streaming E2E toolkit [121]	E2E training recipes	Often CTC + attention 2-pass; streaming variants [122]	Yes (design goal) [123]	High (depends models)	H s s

Model / family	Model or toolkit?	Core architecture	Training paradigm	Decoder style	Streaming support	Offline transcription quality	R s
Vosk	Toolkit + pretrained models	Kaldi-based offline ASR toolkit with streaming API [125]	Traditional ASR models; toolkit distribution	WFST-style decoding (Kaldi heritage) [126]	Yes (streaming API) [127]	Mixed (model-dependent)	G s e

B. Compact decoder table: ASR architecture patterns

Pattern	What it is	Best at	Weak at	Streaming fit	Latency implications	Deployment implications
CTC	Framewise symbol prediction with alignment marginalization [129]	Simple decoding; low latency; easy beam search	Conditional independence limits; may need LM/ rescoring	Good	Low; can do partial results	Often easiest to export/ quantize; great for edge
RNN-T / transducer	Encoder + predictor + joint network; outputs conditioned on past tokens [130]	Streaming ASR; better incremental decoding than pure CTC	Complexity; tuning; can be heavier	Excellent (designed for streaming) [131]	Low-to-medium; depends on chunking/ look-ahead	Production streaming staple; more moving parts
Encoder-decoder seq2seq (attention)	Autoregressive decoder attends over encoded audio	Strong offline accuracy; flexible	Streaming is hard without explicit constraints	Weak-to-mixed	Higher; decoding is sequential	More GPU-friendly; less edge-friendly

Pattern	What it is	Best at	Weak at	Streaming fit	Latency implications	Deployment implications
Whisper-like seq2seq	Large seq2seq trained for robustness; tooling uses sequential long-form decoding; turbo shrinks decoder layers [3]	Robust offline transcription; huge ecosystem	True low-latency streaming not native; can hallucinate without VAD	Mixed	Often higher; chunking adds complexity	Great for batch services; edge requires quantization/ runtime
Conformer	Transformer + convolution to capture local+global dependencies [132]	Strong ASR encoder backbone; robust	Heavier than small CNN/ RNN; needs careful streaming design	Good with streaming design	Medium; can be cache-aware streaming [120]	Widely used in production ASR (CTC/ RNN-T heads)
wav2vec2 SSL + head	SSL encoder pretrain + finetune CTC head [133]	Data efficiency; transfer	Latency depends on encoder; not inherently streaming	Mixed	Medium	Great for domain adaptation; export varies
HuBERT / XLS-R / WavLM SSL	SSL with masked prediction / clustering / denoising; large multilingual variants [134]	Transfer across tasks; multilingual (XLS-R)	Often heavy; not “plug-and-play streaming”	Mixed	Medium-high	Strong backbone for many tasks (ASR, emotion, speaker), but needs finetune

C. Use-case fit table (best / acceptable / poor)

Use case	Best fits	Acceptable	Poor	Reason
Offline transcription (general)	faster-whisper; Whisper; WhisperX-style batching (if timestamps needed) [135]	SpeechBrain ASR recipes; ESPnet/WeNet models [136]	Pure streaming stacks tuned for latency only	Offline wants accuracy + throughput; batching dominates costs [137]

Use case	Best fits	Acceptable	Poor	Reason
Streaming captions	NeMo cache-aware streaming Conformer/ FastConformer; Sherpa transducer; WeNet streaming setups [138]	Vosk (simpler) [127]	Whisper-family baseline (without heavy chunking/hacks) [3]	Streaming needs bounded look-ahead + stable partials [120]
Embedded/ offline device	whisper.cpp (quantized); sherpa-ncnn; Vosk small models [139]	sherpa-onnx (if model available) [140]	Large seq2seq models in FP16	Memory + thermal constraints
Call-center speech	Domain-tuned streaming Conformer/ RNN-T stacks; NeMo diarization pipeline if needed [82]	Whisper with strong VAD + domain adaptation	Generic models without domain adaptation	Telephony noise + overlap + jargon
Noisy speech	Whisper-family often robust; Conformer models with robust training; Distil-Whisper claims robustness retention (English) [141]	SSL finetune (wav2vec2/ WavLM) [142]	Small keyword spotters	Noise robustness depends on training
Multilingual speech	Whisper-family; XLS-R finetuning pipelines [143]	Some NeMo multilingual models	Monolingual domain models	Coverage vs accuracy tradeoffs
Russian speech	GigaAM/GigaAM-v3; Russian NeMo models; Vosk RU models (lighter) [144]	Whisper multilingual (mixed) [10]	English-only distill models	Russian-specific training matters [10]
Long-form audio	Whisper-family sequential decoding; WhisperX pipeline for time-accurate segmentation [145]	Chunked streaming ASR with careful stitching	Naive streaming chunking without endpointing	Drift + segmentation + hallucination issues [146]
Low-latency command recognition	CTC small models; Vosk small; dedicated KWS	RNN-T small with tiny look-ahead	Whisper seq2seq	Needs sub-100ms and stable endpointing [147]

D. Short practical notes (minimal prose)

Why streaming ASR is separate: training/inference must match bounded right-context and caching; otherwise streaming WER and stability can degrade. [148]

Why Whisper-like models are influential but not ideal for low-latency streaming: autoregressive decoding + large receptive context + long-form heuristics; “turbo” improves speed by shrinking decoder layers but remains seq2seq. [3]

Why Conformer + CTC/RNN-T remains production-important: strong encoder backbone with streaming-compatible heads; cache-aware streaming is explicitly supported in some production toolkits. [149]

Common confusion / misconceptions
"RNN-T exists" ≠ "your pipeline is streaming-ready" (endpointing and partial results matter). [150]
"Turbo" is not a general translation model; maintainers explicitly note they don't expect it to perform well on translation data. [3]

Speech emotion recognition

A. Main emotion table

Name	Type	Primary task or secondary capability?	Architecture	Input granularity	Emotion formulation	Self-hosted friendly?	Local inference friendly?	CPU-only viable?	Ty h cl
GigaAM-Emo	Task model (fine-tuned)	Primary	Acoustic model fine-tuned from GigaAM encoder [151]	Utterance/segment	Discrete classes (example: 4 emotions in public notes) [152]	Yes	Yes	Yes (but heavy)	C
SpeechBrain wav2vec2-IEMOCAP	Task model	Primary	wav2vec2 base + classifier [154]	Utterance/segment	Categorical (IEMOCAP classes) [155]	Yes	Yes	Yes	C
openSMILE + classifier	Toolkit + features	Secondary (features)	Hand-crafted acoustic feature extraction toolkit [158]	Frame/utterance (feature sets vary)	Any	Yes	Yes	Yes	C

B. Short caution table

Pitfall	Why it matters	How badly it can mislead you	Rule of thumb
Acted datasets (e.g., IEMOCAP)	Prosody differs from real calls/meetings; labels reflect acted intent [156]	High	Treat as “lab benchmark,” not deploy proof [157]
Cross-language transfer	Prosodic and pragmatic cues differ by language/culture [160]	High	Re-train/validate per language/domain
Noisy real-world audio	Noise/reverb shifts features; SER is less robust than ASR	Medium-high	Use strong VAD + denoising + in-domain data if serious
Speaker identity leakage	Models learn speaker traits correlated with labels [157]	High	Evaluate with speaker-disjoint splits; monitor leakage
Domain mismatch	Train/test mismatch collapses reported accuracy [161]	High	Always run in-domain pilot; do not trust cross-paper numbers
Label ambiguity	“Emotion” is subjective and multi-factor	High	Treat outputs as weak signals; use aggregation

C. Practical note (explicit)
Speech emotion recognition is usually **less reliable and more subjective** than ASR; do not operationalize it as ground truth without in-domain validation and clear ethical constraints. [16]

Common confusion / misconceptions
“Model accuracy on IEMOCAP” ≠ “works on customer calls.” [161]
“Positive/negative/neutral” outputs are not psychological diagnoses. [157]

Music and general audio deep dives

Music / MIR

A. Main MIR table

Name	Type	Primary task	Architecture	Input representation	Label style	Self-hosted friendly?	Local inference friendly?
Musicnn	Model/library	Music tagging	CNN on mel spectrogram [71]	Mel spectrogram	Multi-label tags	Yes	Yes

Name	Type	Primary task	Architecture	Input representation	Label style	Self-hosted friendly?	Local inference friendly?
MERT	Representation model	Music embeddings / transfer	Transformer-based music rep model [104]	Spectrogram / learned rep	Embedding (and finetune heads)	Yes	Yes
MusicFM	Representation model	Music embeddings / retrieval	Music foundation representation model [105]	Learned embedding	Embedding	Yes	Yes
CLAP (audio-text)	Embedding model	Text↔audio retrieval; zero-shot tags	Dual encoders + contrastive learning [100]	Audio + text	Embedding	Yes	Yes
PANNs	Model family	Audio tagging; can transfer to music	CNN14/ Wavegram-Logmel CNNs [93]	Log-mel + waveform (some)	Multi-label	Yes	Yes
AST / HTS-AT	Model families	Audio classification; can be adapted to music	Spectrogram transformers [162]	Spectrogram patches	Multi-label	Yes	Yes
Essentia	Toolkit	MIR features (tempo/beat/ melody/etc.)	DSP + some ML models in ecosystem [107]	Time/ frequency features	N/A	Yes	Yes
madmom	Toolkit	Beat/ downbeat/ tempo	Signal processing + RNN processors [163]	Features + RNN activations	N/A	Yes	Yes
BeatNet	Pipeline/model	Beat/ downbeat/ tempo/meter	CRNN + particle filtering modes [165]	Spectrogram/ features	Event times + tempo	Yes	Yes

B. Task-fit table (best / acceptable / poor)

Task	Best fits	Acceptable	Poor	Reason
Auto-tagging large music library	Musicnn; PANNs; lightweight CNN taggers [167]	AST/HTS-AT (GPU batch) [168]	Heavy embedding models without batching	Throughput dominates
Genre classification	Music tagging models finetuned; modern embeddings + classifier [169]	PANNs/AST adapted	“One label = truth” approaches	Genre is fuzzy; dataset bias
Mood/style estimation	Music embeddings (MERT/MusicFM) + calibration [170]	Musicnn tags	Pure AudioSet event taggers	Semantics differ from events
Beat/tempo extraction	Essentia; madmom; BeatNet [171]	librosa beat_track (simple) [172]	CLAP/MERT embeddings alone	Beat is temporal structure task [172]
Instrument recognition	Multi-label taggers trained for instruments; embeddings + head	AudioSet taggers (some overlap)	Beat-only toolkits	Labels depend on dataset
Music similarity / retrieval	MERT/MusicFM embeddings; CLAP for text-driven retrieval [173]	OpenL3 (older) [174]	YAMNet	Music-specific representations matter
Lightweight inference	Musicnn; YAMNet; small PANNs [175]	whisper.cpp-style quantized embeddings (niche)	Large transformers	CPU budget
Research prototyping	SpeechBrain + MIR libs; AST/HTS-AT; AudioMAE/BEATs embeddings [176]	Essentia/ madmom baselines	None	Research wants flexible backbones + baselines

C. Short practical notes (minimal prose)

Spectrogram CNN taggers are often the best “throughput-per-watt” choice for huge catalogs; transformers usually win on representation richness but cost more. [177]

Multi-label tagging ≠ genre classification; genre labels are social constructs and dataset-dependent. [43]

Beat/tempo is an explicitly temporal/structural task; semantic tags and embeddings do not guarantee beat accuracy. [178]

Common confusion / misconceptions

"CLAP can label audio" ≠ "it understands music structure." [179]

"Tempo estimate" ≠ complete rhythm analysis (downbeats/meter/beat positions). [166]

Architecture cheat sheet

Architecture / concept	What it is	Why people use it	Best at	Weak at	Typical tasks	Self-hosting implications	La im
CNN on mel spectrograms	Convs over time-frequency "image"	Fast, strong baseline, easy to deploy	Tagging, simple classification	Long-range structure	Audio tagging, music tags [40]	Easy export; CPU viable	Of su pe (ir
CRNN	CNN features + RNN/GRU temporal modeling	Add temporal smoothing	Beat tracking, SED style tasks	Scaling to long contexts	Beat/ downbeat, frame tasks [180]	Moderate	Ne tir po
Transformer / AST-style	ViT-like attention on spectrogram patches [181]	Strong representation, global context	Audio classification, transfer	Compute heavy	AudioSet classification, transfer	Heavier deps; GPU preferred	Ba of pr
Conformer	Transformer + convolution in encoder [132]	Strong ASR encoder	ASR accuracy/ robustness	Complexity	ASR encoders, streaming variants [182]	Common in production toolkits	St ne ca lo de
wav2vec2-style SSL encoder	Masked/ contrastive SSL on raw audio then finetune [133]	Data efficiency, transfer	ASR with little labeled data	Full pipeline requires finetune	ASR, SER, etc.	Heavy pretrain not needed for inference	La de er siz
HuBERT / XLS-R / WavLM SSL	SSL with clustering targets (HuBERT) / multilingual scale (XLS-R) / denoising (WavLM) [134]	Universal speech representations	Multi-task transfer	Large models heavy	ASR, speaker, emotion, LID	Needs finetune heads	Of ba of
CTC head	Framewise outputs + CTC loss [129]	Simple decoding; streaming-friendly	Low-latency text	Conditional independence	Streaming ASR, alignment	Easier export/ quantize	Lo

Architecture / concept	What it is	Why people use it	Best at	Weak at	Typical tasks	Self-hosting implications	La im
RNN-T / transducer	Streaming seq2seq architecture [183]	Best streaming tradeoff	Streaming ASR	More moving parts	Streaming ASR	Production-proven but more complex	Lo m la
Encoder-decoder seq2seq	Autoregressive decoder with attention	High offline quality	Offline transcription	Streaming	Offline ASR	Harder low-latency	Hi la
Speaker embedding (ECAPA/x-vector)	Fixed-dim voice embedding extractor [184]	Verification/ diarization clustering	Speaker similarity	Needs segmentation	Speaker verification/ diarization	Easy inference; needs pipeline	N ti po (s wi
CLAP-style audio-text embedding	Dual encoders + contrastive alignment [185]	Zero-shot labels, retrieval	Retrieval, promptable tagging	Prompt + dataset dependence	Zero-shot classification, search	Heavier deps; careful packaging	O ba pr

Short practical takeaways (minimal prose)
If you need strict real-time: prefer CTC/RNN-T streaming designs + cache-aware encoders; avoid heavy autoregressive decoding. [186]

Common confusion / misconceptions
“Transformer” ≠ “foundation model”; many transformers are small supervised classifiers. [187]

Tooling and deployment

Toolkit vs model vs family vs wrapper classification

Name	Category	What it actually is	What people often mistake it for	Practical interpretation
Sherpa	Deployment toolkit	Inference framework for CTC/transducer E2E ASR; C++/Python; focuses on deployment not training [56]	“A single ASR model”	Bring your own pretrained model; great if you standardize exports

Name	Category	What it actually is	What people often mistake it for	Practical interpretation
sherpa-onnx	Deployment toolkit + runtime integration	ONNXRuntime-based local inference repo supporting many functions (ASR, diarization, etc.) [57]	“An ASR model”	A multi-task inference bundle; maturity varies by task/model
SpeechBrain	Toolkit (train + inference)	PyTorch speech toolkit with recipes + pretrained models [188]	“A specific SOTA model”	Use it to train/ finetune and to run many pretrained baselines
GigaAM	Model family	Russian foundation model + fine-tuned task models; distributed via package/model hubs [189]	“A turnkey deployment stack”	Strong Russian models; deployment depends on your runtime/export choices
Musicnn	Task model/ library	Music tagging CNN library/models [71]	“General audio embedding foundation model”	Good, fast tag baseline for music
Whisper	Model family	Multilingual seq2seq ASR models; turbo variant shrinks decoder layers for speed [3]	“Streaming ASR model”	Great offline; streaming requires chunking and careful engineering
faster-whisper	Runtime wrapper	Whisper implemented in CTranslate2 for faster inference + quantization [73]	“A better Whisper model”	Mainly a runtime speed/efficiency improvement
whisper.cpp	Runtime + model format	C/C++ inference for Whisper with ggml/gguf quantization [78]	“Whisper model weights”	The practical route to on-device Whisper inference
pyannote	Toolkit + pretrained pipelines	PyTorch diarization toolkit; pipelines often gated on HF; newer community-1 pipeline exists [190]	“Just a diarization model file”	Expect token gating + pipeline components

Name	Category	What it actually is	What people often mistake it for	Practical interpretation
NeMo	Toolkit + model zoo	NVIDIA speech toolkit and model families; includes cache-aware streaming Conformer and diarization pipeline components [191]	“A single ASR model”	A full stack; great if you’re aligned with NVIDIA GPU ecosystem
CLAP	Model family	Contrastive audio-text embedding models (LAION and Microsoft variants exist) [99]	“A universal audio classifier”	It’s an embedding space + prompting; excellent for retrieval/weak labels
YAMNet	Task model	Small audio event classifier trained for AudioSet-style events [91]	“Music tagger”	General event tags; not music-specialized
PANNs	Model family	AudioSet-pretrained CNNs + released code/weights [93]	“One monolithic model”	Choose a specific variant (CNN14, Wavegram-Logmel, etc.)
AST	Model family	Spectrogram transformer classifier [94]	“Audio embedding foundation model”	Often used as backbone; can extract embeddings but compute-heavy
HTS-AT	Model family	Hierarchical transformer + token-semantic module for classification/detection [97]	“Lightweight AST”	Can be more efficient, still heavier than CNNs
BEATs	Representation model	SSL audio transformer representation model [98]	“Audio tagging classifier”	Use it as an embedding backbone; attach heads
MERT	Representation model	Music representation transformer [104]	“A genre classifier”	Use embeddings + downstream heads
MusicFM	Representation model	Music foundation representations [105]	“A turnkey tagging tool”	Use embeddings + your pipeline

Short practical takeaways (minimal prose)
Most “names” in this space are not comparable without categorizing them first: toolkit vs model family vs runtime wrapper. [192]

Common confusion / misconceptions
People often compare “Whisper” (model) to “faster-whisper” (runtime) as if they were different architectures; most differences are inference stack optimizations. [73]

Self-hosted / local inference scorecards

Score levels (qualitative): High / Medium / Low.

A. ASR self-hosting scorecard

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging
Whisper family (baseline impl)	High ecosystem, but baseline runtime is not optimized [3]	Medium (small) / Low (large)	Medium	Low-to-mixed	Low-to-mixed	Medium	Medium
faster-whisper	High [115]	Medium-high (int8/small)	High (CTranslate2) [193]	Mixed (still Whisper)	Medium	High	High
whisper.cpp	High for local/edge [78]	High (core purpose)	Mixed	Mixed (chunking)	High	Medium	Medium
Sherpa / sherpa-onnx	Medium-high (deployment-focused) [194]	Medium (model-dependent)	Medium-high (onnxruntime/CUDA EP possible) [195]	High when using streaming-native models	High (sherpa-ncnn exists for mobile) [196]	Medium	Medium-high

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging
GigaAM / GigaAM-v3	Medium (models are available; deployment paths vary) [197]	Medium (heavy)	Medium-high	Medium (RNN-T exists) [10]	Low-to-medium	Medium	Medium
NeMo ASR families	High (but heavy stack) [120]	Low-to-medium	High on NVIDIA GPUs	High (cache-aware streaming) [84]	Low	Medium	Medium
SpeechBrain ASR	Medium (research-first) [188]	Medium (small)	Medium	Mixed	Low-to-medium	Medium	Medium

B. Emotion self-hosting scorecard

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging	Common deployment trap
SpeechBrain wav2vec2 SER	High (easy inference), low reliability	High	Medium	Clip-level yes	Medium	High	High	Using as ground truth without in-domain validation [16]
GigaAM-Emo	Medium	Medium	Medium	Clip-level yes	Low-to-medium	Medium	Medium	Overtrusting benchmark gains; dataset mismatch [198]

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging	Common deployment trap
openSMILE baseline	High	High	N/A	Yes	High	Medium	Medium	Thinking it' "modern SER" rather than features + ML

C. MIR self-hosting scorecard

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging	Batch scanning friendliness
Musicnn	High	High	Medium	Clip-level yes	Medium	High	High	High
YAMNet	High	High	N/A	Frame/clip yes	High	High	High	High
PANNs	High	High	Medium	Clip-level yes	Medium	Medium	Medium	High
AST / HTS-AT	Medium	Medium-low	High	Clip-level (not streaming focus)	Low-to-medium	Medium	Medium	Medium
BEATs	Medium	Medium-low	High	Batch embeddings	Low-to-medium	Medium	Medium	Medium
CLAP	Medium	Medium-low	High	Batch embeddings	Low-to-medium	Medium	Medium	Medium

Stack	Self-hosted maturity	CPU-only viability	GPU efficiency	Streaming viability	Embedded / ARM viability	Docker friendliness	Ease of packaging	Batch scanning friendliness
MERT	Medium	Medium-low	High	Batch embeddings	Low-to-medium	Medium	Medium	Medium
MusicFM	Medium	Medium-low	High	Batch embeddings	Low-to-medium	Medium	Medium	Medium
Essentia	High	High	N/A	Yes (many ops)	High	Medium	Medium	High

Short practical takeaways (minimal prose)
For local ASR, the "winning" stack is usually one of: CTranslate2-based (faster-whisper), ggml/gguf-based (whisper.cpp), or ONNXRuntime-based (sherpa-onnx)—not bare PyTorch. [199]
For diarization, the largest non-ML friction is gated model access + packaging; plan for that early. [200]

Common confusion / misconceptions
"Docker-friendly" doesn't guarantee predictable latency: audio pipelines can be IO-bound (decode, resample, VAD). [201]

Hardware & latency guide

A. Hardware class table

Hardware class	What kinds of models fit	Example tasks	Caveats
Tiny CPU-friendly	YAMNet; small CNN taggers; small Vosk models; openSMILE features [202]	Event tagging; keyword/command ASR; basic MIR	Limited accuracy vs large models
Laptop CPU acceptable	Musicnn; PANNs small variants; whisper.cpp small/quantized [203]	Library tagging; offline transcription (slow)	Sustained throughput limited
Desktop CPU acceptable	whisper.cpp quantized medium-ish; Vosk larger models [204]	Offline STT at moderate speed	Memory bandwidth matters

Hardware class	What kinds of models fit	Example tasks	Caveats
Commodity GPU (6–8GB VRAM)	faster-whisper medium/large-v3-turbo; pyannote diarization practical setups [205]	Batch transcription; diarization	Need CUDA/cuDNN/driver match
Mid GPU (10–16GB VRAM)	Large Whisper variants; AST/HTS-AT; BEATs/CLAP embeddings at throughput [206]	High-throughput transcription; embeddings	IO + preprocessing still bottleneck
Larger GPU (16–24GB+)	Bigger batch sizes; multi-stage pipelines; training/finetune	High-scale services	Engineering complexity
ARM SBC / edge device	sherpa-ncnn; Vosk small models; whisper.cpp small/quantized [207]	On-device commands; small tagging	Thermal + limited RAM
NPU/mobile accelerator	ONNXRuntime EPs + vendor stacks (varies) [208]	Phone deployments	Operator support limits; quantization constraints

B. Latency / throughput classification table

Scenario	Preferred model classes	Acceptable	Avoid	Why
Ultra-low-latency command recognition	Small CTC/KWS models; Vosk small	Small RNN-T	Whisper seq2seq	Strict latency + endpointing
Near-real-time streaming ASR	Cache-aware streaming Conformer/RNN-T; sherpa transducer [209]	Vosk streaming API [127]	Offline seq2seq without streaming design	Bounded right context
Real-time diarization	Specialized streaming diarization (niche); otherwise constrained pipelines	NeMo diarization with tuning	Heavy batch diarization default configs	Overlap + clustering costs
Batch transcription	faster-whisper; WhisperX-style batching [135]	Whisper baseline	Streaming ASR stacks (if accuracy lower)	Throughput dominates
Large-library music tagging	Musicnn; PANNs; lightweight taggers [167]	AST/HTS-AT on GPU	Heavy embeddings without batching	Millions of clips require throughput

Scenario	Preferred model classes	Acceptable	Avoid	Why
Clip-level emotion inference	SSL backbone + classifier	openSMILE baseline	Any “emotion as truth” system	Reliability is limiting factor [16]

C. IMPORTANT (explicit)
Where exact latency data is sparse, treat performance as classes (excellent / near-real-time with tuning / batch preferred); do not assume reported “xRT” applies to your audio lengths, beams, and hardware. Example benchmarks exist for some stacks (e.g., faster-whisper turbo memory/time reports), but they are configuration-specific. [210]

Common confusion / misconceptions
“GPU recommended” is not the same as “needs a massive GPU”; many inference stacks run well on 6–8GB for practical sizes if quantized and batched. [205]

Decision guides

What matters vs noise

Label / concept	Usually important?	Why it matters	Common misunderstanding	Practical rule of thumb
Streaming	Yes	Changes model design + decoding + UX constraints [209]	“Any ASR can stream”	Pick streaming-native architectures/toolkits if latency matters
Multilingual	Yes	Coverage vs accuracy tradeoffs; data matters [143]	“Multilingual = strong on every language”	Validate per language/domain
Conformer-based	Often	Strong encoder backbone across ASR [182]	“Conformer = streaming”	Streaming requires constrained right context + caching [120]
Whisper-compatible	Sometimes	Ecosystem advantage; tooling	“Compatibility implies streaming/low-latency”	Use for offline/batch unless you engineer streaming
CTC	Often	Simple/fast decoding; good for streaming	“CTC is always worse”	With good encoder + LM/rescore, can be strong
RNN-T	Often (streaming)	Production streaming staple [131]	“Too complex to deploy”	Use mature toolkits (NeMo/Sherpa)

Label / concept	Usually important?	Why it matters	Common misunderstanding	Practical rule of thumb
Seq2seq	Context-dependent	Strong offline quality	“Best overall”	Avoid for strict latency unless designed for it
SSL pretraining	Often	Improves transfer/low-resource performance [211]	“SSL solves domain shift”	Still need in-domain finetune/validation
Emotion recognition	Usually not as “core truth”	Subjective; brittle across domains [16]	“Objective affect detector”	Use as weak signal; calibrate
Diarization	Often	Required for multi-speaker transcripts	“Speakers are named identities”	It outputs session-local labels; ID is separate [13]
Music tagging	Often	Multi-label semantics for catalogs	“Genre truth”	Use as probabilistic metadata
Audio embeddings	Often	Enables retrieval/transfer	“One embedding fits all”	Choose embedding family per domain (speech vs music vs events) [212]
Zero-shot	Sometimes	Reduces labeling	“No calibration needed”	Prompts and label definitions dominate outcomes [213]
Real-time	Yes	Latency budgets drive architecture choices	“Real-time = fast GPU”	Real-time is end-to-end pipeline + buffering
On-device	Yes	Memory/thermal constraints	“Quantization always free”	Quantization trades accuracy for speed/size [214]
Benchmark score	Sometimes	Useful for sanity	“Leaderboard = deployable”	Prefer deployment maturity + domain validation
“Foundation model”	Sometimes	Signals transfer potential	“Foundation = best at everything”	Treat as representation; attach task heads and evaluate
“State-of-the-art”	Usually not	Often dataset-specific and time-sensitive	“SOTA means best in my domain”	Only trust when dataset/domain matches

Short practical takeaways (minimal prose)
The single most predictive question for deployment success is: “Does this stack include a proven inference runtime + quantization + batching for my hardware?” [215]

Common confusion / misconceptions
“Foundation” and “SOTA” are not deployment properties; they’re research labels. [216]

Shortlists and conclusions

Final shortlists

A. Best current ASR entries by scenario

Scenario	Shortlist entry	One-line description	Why it matters	Strongest use case	Main caveat
Best open offline ASR baseline	faster-whisper + Whisper (large-v3 / turbo) [217]	Optimized Whisper inference via CTranslate2	Best practical speed/quality balance	Batch transcription services	Still not true streaming
Best streaming-friendly open stack	NeMo cache-aware streaming Conformer/ FastConformer [84]	Streaming-first ASR with bounded right context + caching	Predictable latency design	Live captions/ calls	Heavy stack; GPU-centric
Best CPU-friendly local ASR	whisper.cpp (quantized models) [78]	Local Whisper inference with ggml/ gguf quantization	Works on commodity CPUs/ARM	On-device/ offline	Accuracy vs quantization; pipeline glue needed
Best embedded/ edge-friendly ASR	sherpa-ncnn (streaming) / Vosk small models [218]	Mobile-focused inference paths	Real edge deployability	Offline commands, simple STT	Model quality varies
Best Russian ASR	GigaAM-v3 (CTC/RNNT variants) [118]	Russian-specialized foundation + finetuned ASR	Strong Russian benchmarks vs general models	Russian production STT	License constraints; deployment stack needed

Scenario	Shortlist entry	One-line description	Why it matters	Strongest use case	Main caveat
Best long-form transcription choice	Whisper + WhisperX-style segmentation/ alignment pipeline [219]	Adds VAD + alignment for better timestamps	Better timing and long-form handling	Podcasts/ meetings	Heavier pipeline; diarization may require gated models

B. Best current speech emotion options

Scenario	Shortlist entry	One-line description	Why it matters	Strongest use case	Main caveat
Best open experimentation baseline	SpeechBrain wav2vec2-IEMOCAP [220]	Common SER reference model	Easy baselines + reproducibility	Rapid SER experiments	Poor domain generalization [16]
Best self-hosted practical option	openSMILE + simple classifier [158]	Feature-based baseline	Fast, interpretable	Lightweight metadata	Not modern SER; limited accuracy ceiling
Best "good enough but caution"	GigaAM-Emo (Russian) [153]	Russian SER from Russian foundation model	Best bet if you must do Russian SER	In-domain Russian analysis	Still subjective/ brittle

C. Best current music / MIR options

Scenario	Shortlist entry	One-line description	Why it matters	Strongest use case	Main caveat
Best open music tagging baseline	Musicnn [71]	Fast CNN music tagging	High throughput	Large library tagging	Aging vs transformer reps
Best lightweight large-library scanner	PANNs (CNN14 family) [93]	Strong AudioSet-pretrained tag baseline	Practical + scalable	Broad tags for big catalogs	Not music-specialized
Best audio embedding baseline	BEATs (general audio) [98]	Strong general-purpose audio embeddings	Transfer to many tasks	Retrieval/ classification	Compute heavy

Scenario	Shortlist entry	One-line description	Why it matters	Strongest use case	Main caveat
Best research-first modern representation model	MERT or MusicFM [221]	Music-specialized embeddings	Better music similarity/transfer	Similarity/search	Requires heads/pipeline; evolving ecosystem

Short practical takeaways (minimal prose)
Pick the shortlist by *deployment constraint first* (streaming vs offline vs edge) and *domain second* (Russian vs multilingual). Model architecture comes third. [222]

Common confusion / misconceptions
"Best offline model" is often a bad streaming choice. [30]

Key conclusions

Conclusion	What to do immediately	Why
Separate model family from runtime wrapper from toolkit	Maintain a small internal taxonomy (Model / Toolkit / Runtime / Pipeline) and tag every dependency accordingly	Prevents "apples to oranges" design decisions
For real-time ASR, prioritize streaming-native designs	Use cache-aware streaming Conformer/RNN-T stacks or transducer-focused deployment frameworks [223]	Latency + stability come from architecture + decoding constraints
For offline transcription, optimize inference stack first	Prefer CTranslate2-based or ggml/gguf-based inference before changing model families [224]	Quantization + batching dominate cost
For Russian ASR, use Russian-trained models	Start with GigaAM-v3 family; compare against multilingual baselines on your domain [118]	Language/domain specialization matters more than generic multilingual coverage
Treat SER as weak metadata	Use only with strong in-domain validation and careful UX/ethics [16]	High subjectivity + dataset bias
MIR requires separating semantics vs rhythm	Use separate tools/models for semantic tagging vs beat/tempo extraction [225]	Structural rhythm tasks are different from semantic labeling

Common confusion / misconceptions
A "single best model" across ASR + diarization + emotion + MIR rarely exists; production systems are modular pipelines with multiple specialized components. [226]

(First mention of organizations for context, only once each): OpenAI [227] NVIDIA [228] Microsoft [229] Tencent [230] LAION [231] Hugging Face [232] Meta AI [233] SberDevices [234]

---

[1] [2] [73] [74] [76] [77] [115] [135] [199] [217] [224] https://pypi.org/project/faster-whisper/
https://pypi.org/project/faster-whisper/

[3] [25] [30] [72] [141] [143] [145] [234] https://github.com/openai/whisper/discussions/2363
https://github.com/openai/whisper/discussions/2363

[4] [23] [29] [50] [55] [56] [192] [194] https://github.com/k2-fsa/sherpa
https://github.com/k2-fsa/sherpa

[5] [196] [207] [218] https://github.com/k2-fsa/sherpa-ncnn
https://github.com/k2-fsa/sherpa-ncnn

[6] [7] [78] [79] [80] [139] [204] https://github.com/ggml-org/whisper.cpp
https://github.com/ggml-org/whisper.cpp

[8] [63] [64] https://huggingface.co/niobures/GigaAM
https://huggingface.co/niobures/GigaAM

[9] [61] [197] https://pypi.org/project/gigaam/
https://pypi.org/project/gigaam/

[10] [144] https://huggingface.co/niobures/GigaAM/resolve/main/README_ru.md?download=true
https://huggingface.co/niobures/GigaAM/resolve/main/README_ru.md?download=true

[11] [12] [13] [37] [54] [88] [200] https://huggingface.co/pyannote/speaker-diarization-3.1
https://huggingface.co/pyannote/speaker-diarization-3.1

[14] [32] https://github.com/salute-developers/GigaAM
https://github.com/salute-developers/GigaAM

[15] [154] [155] [220] [229] https://huggingface.co/speechbrain/emotion-recognition-wav2vec2-IEMOCAP
https://huggingface.co/speechbrain/emotion-recognition-wav2vec2-IEMOCAP

[16] [160] [161] https://arxiv.org/html/2406.09933v1
https://arxiv.org/html/2406.09933v1

[17] [19] [27] [40] [49] [92] [93] https://arxiv.org/abs/1912.10211
https://arxiv.org/abs/1912.10211

[18] [38] [39] [90] [91] [202] https://github.com/minzwon/sota-music-tagging-models/blob/master/README.md
https://github.com/minzwon/sota-music-tagging-models/blob/master/README.md

[20] [21] [104] [169] [170] [173] [221] https://github.com/pxaris/FM-music-tagging
https://github.com/pxaris/FM-music-tagging

[22] [107] [108] [109] [171] https://essentia.upf.edu/
https://essentia.upf.edu/

[24] [111] [193] [215] https://github.com/opennmt/ctranslate2
https://github.com/opennmt/ctranslate2

26 35 36 https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/speaker_diarization/intro.html
https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/speaker_diarization/intro.html

28 132 149 182 https://arxiv.org/abs/2005.08100
https://arxiv.org/abs/2005.08100

31 47 133 142 211 https://arxiv.org/abs/2006.11477
https://arxiv.org/abs/2006.11477

33 184 https://www.isca-archive.org/interspeech_2020/desplanques20_interspeech.html
https://www.isca-archive.org/interspeech_2020/desplanques20_interspeech.html

34 https://huggingface.co/speechbrain/spkrec-ecapa-voxceleb
https://huggingface.co/speechbrain/spkrec-ecapa-voxceleb

41 42 43 70 71 167 175 177 203 https://github.com/minzwon/musicfm
https://github.com/minzwon/musicfm

44 45 46 172 178 225 https://librosa.org/doc/main/generated/librosa.beat.beat_track.html
https://librosa.org/doc/main/generated/librosa.beat.beat_track.html

48 98 YuanGongND/ast: Code for the Interspeech 2021 paper " ...
https://github.com/YuanGongND/ast?utm_source=chatgpt.com

51 52 66 67 68 69 136 176 188 233 https://github.com/speechbrain/speechbrain
https://github.com/speechbrain/speechbrain

53 112 137 201 226 https://github.com/m-bain/whisperx
https://github.com/m-bain/whisperx

57 60 113 https://github.com/k2-fsa/sherpa-onnx
https://github.com/k2-fsa/sherpa-onnx

58 59 116 140 https://k2-fsa.github.io/sherpa/onnx/index.html
https://k2-fsa.github.io/sherpa/onnx/index.html

62 118 189 https://huggingface.co/ai-sage/GigaAM-v3/blob/main/README.md
https://huggingface.co/ai-sage/GigaAM-v3/blob/main/README.md

65 https://sberdevices.ru/press/detail/sber-devices-predstavila-giga-am-semejstvo-open-source-modelej-mashinnogo-obucheniya-dlya-raspoznavaniya-rechi-i-emoczij/
https://sberdevices.ru/press/detail/sber-devices-predstavila-giga-am-semejstvo-open-source-modelej-mashinnogo-obucheniya-dlya-raspoznavaniya-rechi-i-emoczij/

75 205 206 210 https://github.com/SYSTRAN/faster-whisper/issues/1030
https://github.com/SYSTRAN/faster-whisper/issues/1030

81 82 84 85 119 120 138 148 186 191 209 222 223 https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/models.html
https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/models.html

83 https://huggingface.co/nvidia/stt_en_fastconformer_hybrid_large_streaming_multi/blob/main/README.md
https://huggingface.co/nvidia/stt_en_fastconformer_hybrid_large_streaming_multi/blob/main/README.md

86 87 190 https://github.com/pyannote/pyannote-audio
https://github.com/pyannote/pyannote-audio

89 https://github.com/pyannote/pyannote-audio/blob/main/LICENSE
https://github.com/pyannote/pyannote-audio/blob/main/LICENSE

94 https://github.com/YuanGongND/ast
https://github.com/YuanGongND/ast

95 https://github.com/huggingface/transformers/blob/main/docs/source/en/model_doc/audio-spectrogram-transformer.md
https://github.com/huggingface/transformers/blob/main/docs/source/en/model_doc/audio-spectrogram-transformer.md

96 97 168 https://arxiv.org/abs/2202.00874
https://arxiv.org/abs/2202.00874

99 100 101 179 227 https://arxiv.org/abs/2211.06687
https://arxiv.org/abs/2211.06687

102 https://github.com/LAION-AI/CLAP
https://github.com/LAION-AI/CLAP

103 114 https://arxiv.org/abs/2506.23553
https://arxiv.org/abs/2506.23553

105 106 https://arxiv.org/html/2411.19371v1
https://arxiv.org/html/2411.19371v1

110 https://www.justinsalamon.com/uploads/4/3/9/4/4394963/bogdanov_essentia_ismir13.pdf
https://www.justinsalamon.com/uploads/4/3/9/4/4394963/bogdanov_essentia_ismir13.pdf

117 https://k2-fsa.github.io/sherpa/ncnn/android/build-sherpa-ncnn.html
https://k2-fsa.github.io/sherpa/ncnn/android/build-sherpa-ncnn.html

121 124 https://github.com/wenet-e2e/wenet
https://github.com/wenet-e2e/wenet

122 123 https://www.isca-archive.org/interspeech_2021/yao21_interspeech.html
https://www.isca-archive.org/interspeech_2021/yao21_interspeech.html

125 https://github.com/alphacep/vosk-api
https://github.com/alphacep/vosk-api

126 https://github.com/kaldi-asr/kaldi
https://github.com/kaldi-asr/kaldi

127 https://alphacephei.com/vosk/
https://alphacephei.com/vosk/

128 https://alphacephei.com/vosk/models
https://alphacephei.com/vosk/models

129 147 https://www.cs.toronto.edu/~graves/icml_2006.pdf
https://www.cs.toronto.edu/~graves/icml_2006.pdf

130 https://www.cs.toronto.edu/~graves/icassp_2013.pdf
https://www.cs.toronto.edu/~graves/icassp_2013.pdf

131 150 183 https://docs.nvidia.com/dl-cuda-graph/examples/rnnt.html
https://docs.nvidia.com/dl-cuda-graph/examples/rnnt.html

134 https://arxiv.org/abs/2106.07447
https://arxiv.org/abs/2106.07447

146 219 https://arxiv.org/html/2303.00747v2
https://arxiv.org/html/2303.00747v2

151 153 198 https://huggingface.co/niobures/GigaAM/blob/main/README.md
https://huggingface.co/niobures/GigaAM/blob/main/README.md

152 https://habr.com/ru/articles/935802/
https://habr.com/ru/articles/935802/

156 https://www.isca-archive.org/smm_2024/emmanouilidou24_smm.pdf
https://www.isca-archive.org/smm_2024/emmanouilidou24_smm.pdf

157 https://openaccess.thecvf.com/content/CVPR2023W/ABAW/papers/Palotti_Analysis_of_Emotion_Annotation_Strength_Improves_Generalization_in_Speech_Emotion_CVPRW_2023_paper.pdf
https://openaccess.thecvf.com/content/CVPR2023W/ABAW/papers/Palotti_Analysis_of_Emotion_Annotation_Strength_Improves_Generalization_in_Speech_Emotion_CVPRW_2023_paper.pdf

158 https://github.com/audeering/opensmile
https://github.com/audeering/opensmile

159 https://www.audeering.com/research/opensmile/
https://www.audeering.com/research/opensmile/

162 181 187 230 232 https://huggingface.co/docs/transformers/en/model_doc/audio-spectrogram-transformer
https://huggingface.co/docs/transformers/en/model_doc/audio-spectrogram-transformer

163 https://github.com/cpjku/madmom
https://github.com/cpjku/madmom

164 https://arxiv.org/abs/1605.07008
https://arxiv.org/abs/1605.07008

165 166 228 https://github.com/mjhydri/BeatNet
https://github.com/mjhydri/BeatNet

174 https://github.com/marl/openl3
https://github.com/marl/openl3

180 https://madmom.readthedocs.io/en/v0.16/modules/features/beats.html
https://madmom.readthedocs.io/en/v0.16/modules/features/beats.html

185 213 231 https://arxiv.org/abs/2206.04769
https://arxiv.org/abs/2206.04769

195 https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html
https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html

208 https://onnxruntime.ai/docs/execution-providers/
https://onnxruntime.ai/docs/execution-providers/

212 https://arxiv.org/abs/2110.13900
https://arxiv.org/abs/2110.13900

214 https://opennmt.net/CTranslate2/quantization.html
https://opennmt.net/CTranslate2/quantization.html

216 https://arxiv.org/html/2501.15177v1
https://arxiv.org/html/2501.15177v1